B 2.4.2 Searching Python

B 2.4.2 Construct and trace algorithms to implement a linear search and a binary search for data retrieval.

Dive into the world of search algorithms. This section explores linear and binary search, comparing their efficiency and guiding you in choosing the right one for your needs. Learn how to implement and trace these algorithms in Java and Python, and discover how data organization impacts search efficiency in real-world scenarios.

Introduction

Searching for specific data within a collection is a fundamental operation in computer science. In this chapter, we'll explore two essential search algorithms: linear search and binary search. We'll learn how they work, how to implement them in Python, and how to analyze their efficiency.

Linear Search

Linear search is a simple algorithm that checks each element in a list sequentially until the target value is found or the end of the list is reached. It's like looking for a book in a bookshelf by starting at one end and checking each book one by one.

Implementation

```
def linear_search(my_list, target):
  """
  Performs a linear search on a list.

  Args:
    my_list: The list to be searched.
    target: The value to search for.

  Returns:
    The index of the target if found, -1 otherwise.
  """
  for i in range(len(my_list)):
    if my_list[i] == target:
      return i  # Found the target at index i
  return -1  # Target not found
```

Tracing the Algorithm

Let's trace the execution of the linear_search function for the list [10, 5, 8, 12, 15] and target = 8:

1. The loop starts at index 0.
2. my_list[0] is 10, which is not equal to 8.
3. The loop continues to index 1.
4. my_list[1] is 5, which is not equal to 8.
5. The loop continues to index 2.
6. my_list[2] is 8, which is equal to the target.
7. The function returns the index 2.

Efficiency Analysis

The time complexity of linear search is O(n), where n is the number of elements in the list. This means that in the worst-case scenario (target not found or at the end of the list), the algorithm needs to check all the elements.

Binary Search

Binary search is a much more efficient algorithm for searching a sorted list. It works by repeatedly dividing the search interval in half. If the target value is less than the middle element, the search continues in the left half. If the target value is greater, the search continues in the right half.

Implementation

```
def binary_search(my_list, target):
  """
  Performs a binary search on a sorted list.

  Args:
    my_list: The sorted list to be searched.
    target: The value to search for.

  Returns:
    The index of the target if found, -1 otherwise.
  """
  low = 0
  high = len(my_list) - 1
  while low <= high:
    mid = (low + high) // 2  # Use // for integer division in Python
    if my_list[mid] == target:
      return mid  # Found the target at index mid
    elif my_list[mid] < target:
      low = mid + 1  # Search in the right half
    else:
      high = mid - 1  # Search in the left half
  return -1  # Target not found
```

Tracing the Algorithm

Let's trace the execution of the binary_search function for the sorted list [2, 5, 7, 10, 11, 14, 15] and target = 11:

1. low = 0, high = 6, mid = 3.
2. my_list[3] is 10, which is less than 11, so low = 4.
3. low = 4, high = 6, mid = 5.
4. my_list[5] is 14, which is greater than 11, so high = 4.
5. low = 4, high = 4, mid = 4.
6. my_list[4] is 11, which is equal to the target.
7. The function returns the index 4.

Efficiency Analysis

Binary search has a time complexity of O(log n), which makes it significantly faster than linear search for large datasets.

Choosing the Right Algorithm

- If the data is unsorted, you have to use linear search.
- Binary search is generally more efficient if the data is sorted, especially for large datasets.
- Consider the trade-off between the overhead of sorting the data and the efficiency gains of using binary search.

Key Questions

Why does binary search require the input list to be sorted?

Answer: Binary search works by comparing the target value to the middle element and then discarding half of the search space. This strategy only works if the list is sorted because the middle element provides information about the relative order of the elements.

If you need to perform many searches on a large dataset, would you prefer a linear search on an unsorted list or a binary search on a sorted list? Justify your answer.

Answer: Binary search on a sorted list would be much more efficient for many searches on a large dataset. Although sorting the data initially takes time, the binary search's O(log n) time complexity will significantly outperform the linear search's O(n) complexity for many searches.

How can you modify the binary search algorithm to find the first occurrence of a target value in a sorted list that may contain duplicates?

Answer: When you find the target value at mid, instead of immediately returning mid, you would need to continue searching in the left half of the list (high = mid - 1) until you find an occurrence of the target where the previous element is not equal to the target. This ensures that you find the first occurrence.